L05: Training Neural Network 1

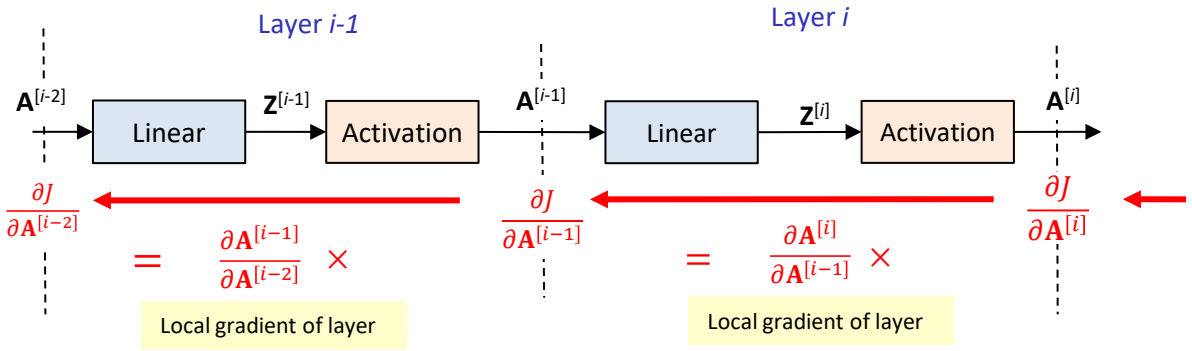
Outline

- Training Neural Network
  - Vanishing and Exploding Gradient
  - Activation Function
  - Initialization
  - Normalization
- Underfitting and Overfitting Issues
  - Regularization
- Hyperparameter Tuning
  - Grid Search
  - Random Search
- Reliability of Reported Performance

Vanishing and Exploding Gradient Issues

Why are deep neural networks difficult to train?

- Gradient instability in deep neural network:

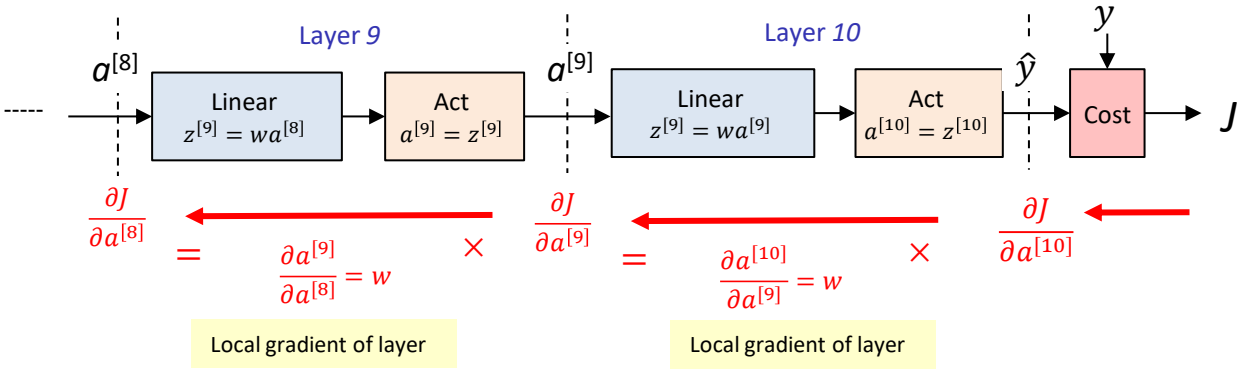


- Local gradient = Jacobian matrix $\mathcal{J}^{[i]} = \frac{\partial \mathbf{A}^{[i]}}{\partial \mathbf{A}^{[i-1]}}$ is a matrix of all partial derivatives describing how the output vector $\mathbf{A}^{[i]}$ changes with the input vector $\mathbf{A}^{[i-1]}$
- Vanishing gradient: If the Jacobian norm $\left\|\mathcal{J}^{[i]}\right\| < 1$ on average across layers, gradients shrink exponentially with depth
- Exploding gradient: If the Jacobian norm $\left\|\mathcal{J}^{[i]}\right\| > 1$ on average across layers, gradients increase exponentially with depth

Why are deep neural networks difficult to train?

- Consider the following network:

- Number of layers $L = 10$
- One neuron per layer,
- Identical weights $w^{[i]} = w$. No bias, $b^{[i]} = 0$
- Identity activation $g(z) = \mathbf{I}(z) = z$
- Number of input feature $F_x = 1$



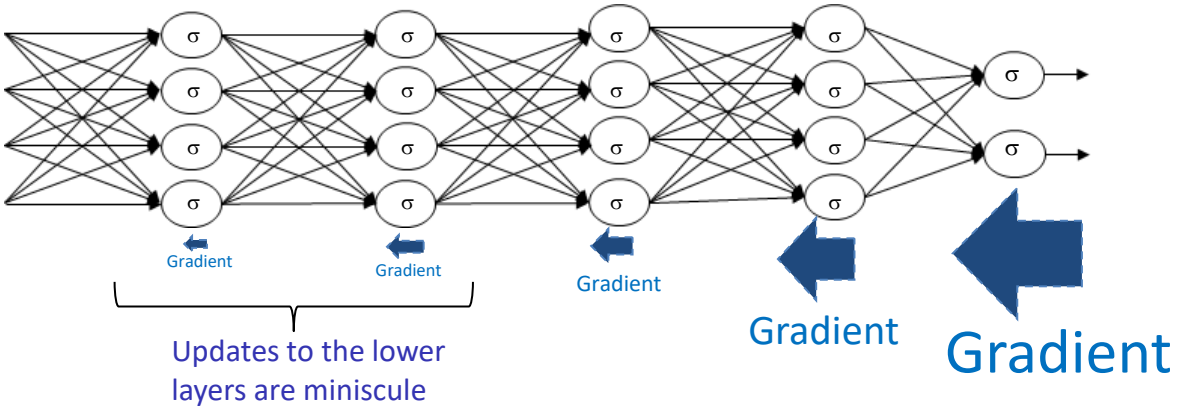
- Local gradient for all layers = $\mathcal{J}^{[i]} = \frac{\partial a^{[i]}}{\partial a^{[i-1]}} = w$
- Norm of the Jacobian $\left\|\mathcal{J}^{[i]}\right\| = |w|$
- Therefore, $\frac{\partial J}{\partial x} = w^{10} \frac{\partial J}{\partial a^{[10]}}$

if $w = 0.2 \rightarrow$ gradient decreases by 0.2^{10} (vanishing gradient effect)

if $w = 5 \rightarrow$ gradient increases by 5^{10} (exploding gradient effect)

Vanishing Gradient Effect

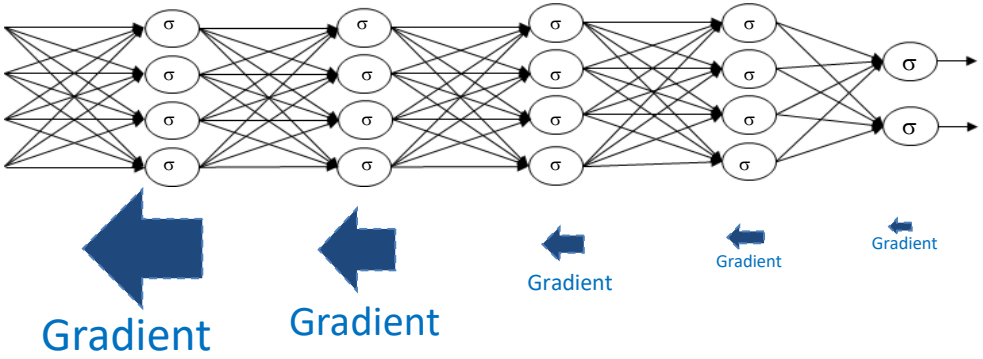
- Vanishing gradient effect: gradients decrease exponentially as they propagate through the layers during backpropagation.



- Early layers learn very slowly or not at all – network struggles to train!
- Factors affecting vanishing gradient issue:
  - Repeated multiplications with layer weights during backprop
  - Poor choice of activation value
  - Poor initialization

Exploding Gradient Effect

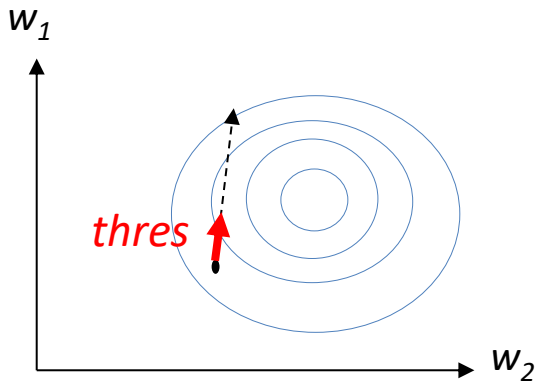
- Exploding gradient effect: gradients increase exponentially as they propagate through earlier layers during backpropagation.



- Cause:
  - Large weights (> 1) leads to exponential growth in gradients
  - High learning rates
  - More severe in RNNs due to repeated multiplications with the same weights
- Solution:
  - Perform gradient clipping (limit gradient magnitude to threshold)
  - Initialize the weights appropriately (Xavier or Kaiming initialization)
  - Use L1 or L2 to penalize large weights
  - Learning rate schedules and adaptive optimizers (Adam, RMSprop)

How to overcome exploding gradient?

- Exploding gradient can be overcome by gradient clipping.
- The gradients are clipped to a threshold during backpropagation. The clipped gradient is used to update the weights.



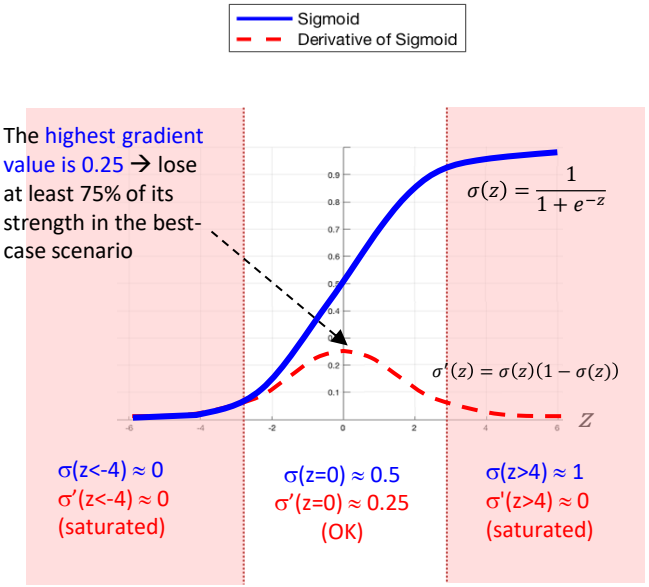
$$\overline{\mathbf{grad}} = \frac{\mathbf{grad}}{\|\mathbf{grad}\|} \, thres$$

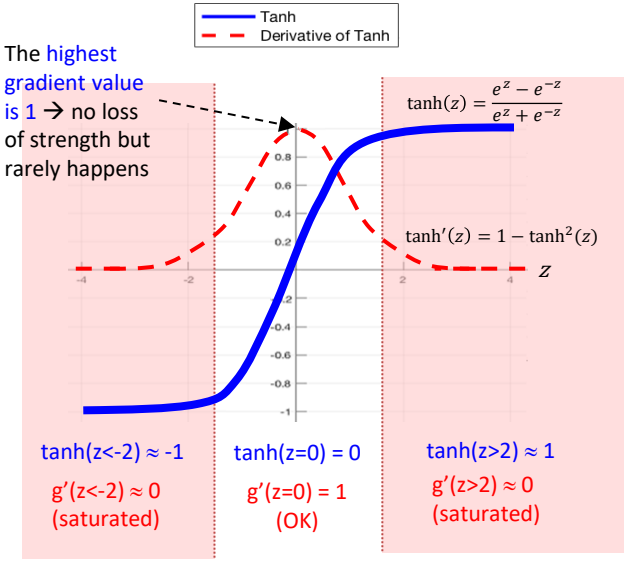
- Parameters are still updated in the same direction.

Activation Function

Impact of activation functions

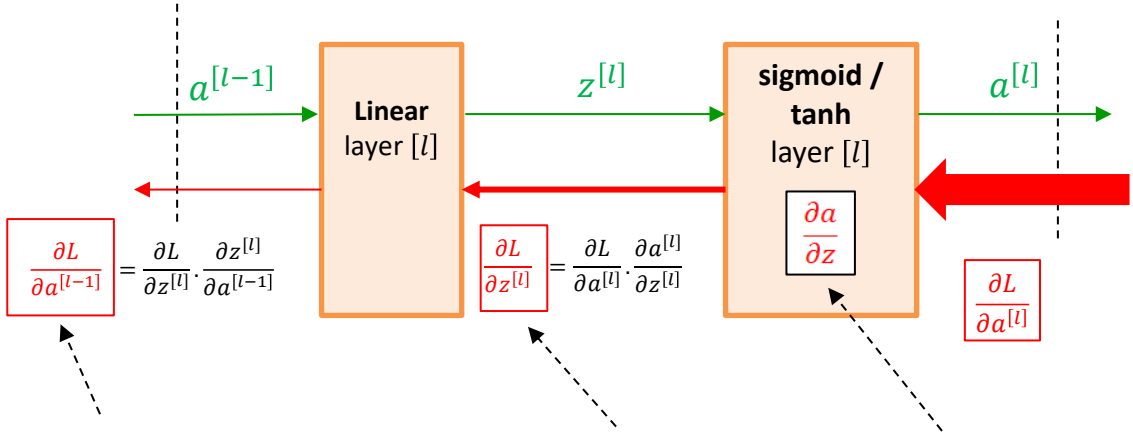
- A deep learning network with sigmoid or tanh activations suffer from vanishing gradient effect.
- A sigmoid and tanh neuron is saturated (local gradient ≈ 0) when it's the activation magnitude is large.



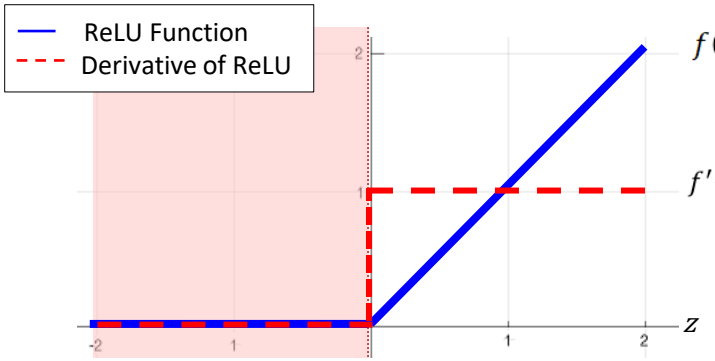


Impact of activation functions

- Backpropagating the gradient through a saturated sigmoid or saturated tanh activation causes all downstream gradient to be very small.



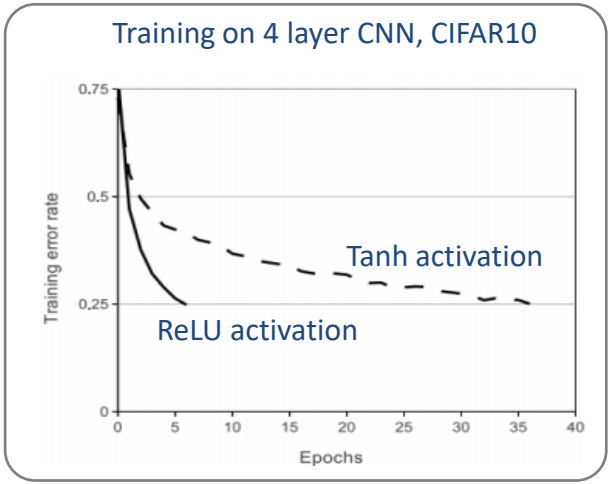
Rectified Linear Unit (ReLU)



$$f(z) = \max(0, z)$$

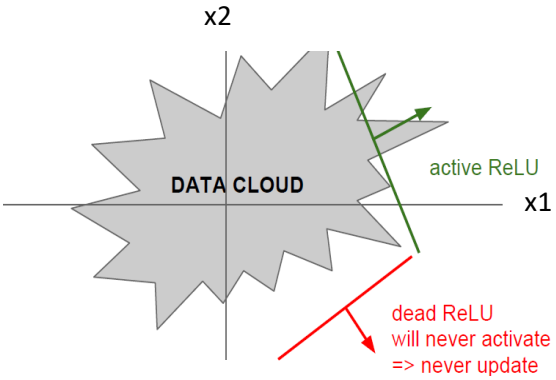
$$f'(z) = \begin{cases} 1 & z > 0 \\ \text{undefined} & z = 0 \\ 0 & z < 0 \end{cases}$$

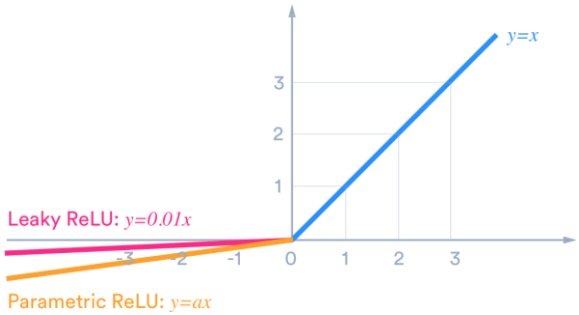
- Recommended for deep neural network
- ☺ Less severe vanishing gradient effect: Does not saturate (in the + region)
- ☺ Computationally efficient. Converges much faster than sigmoid/tanh in practice (6x)



Weakness of ReLU

- ☹ Not-zero centered
- ☹ Still saturated in half of the region
- ☹ Dead ReLUs
  - Dead ReLUs always output 0 and never gets updated
  - Reason: learning rate is too high or big weight update → neuron is saturated for all samples → local gradient is permanently 0.
  - As many as 40% neurons can be dead → lower capacity.
  - Solution: Use smaller learning rate, use Leaky ReLU or Parametric ReLU.

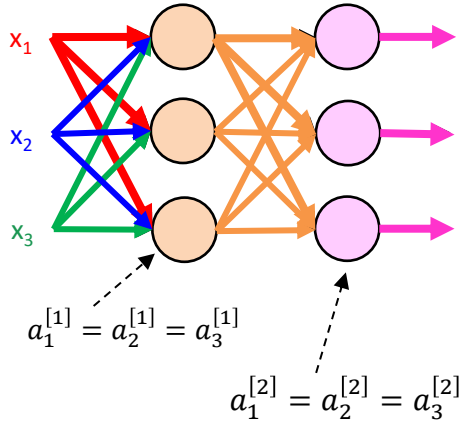


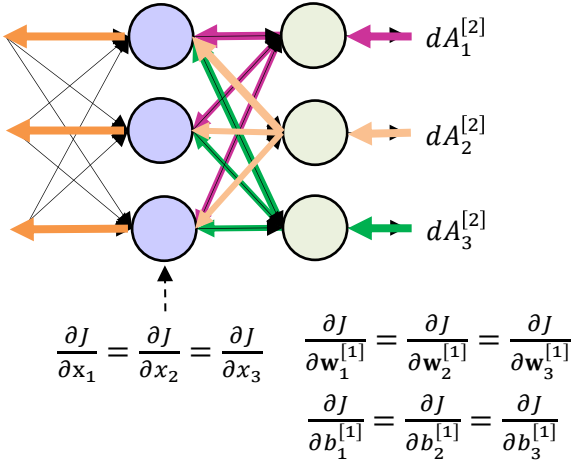


Initialization

Zero Initialization

- Initializes all weights to zero.
- When all weights are initialized identically (e.g., zero), this causes symmetry problem – every neurons in the layer compute the same output and receive the same gradient during backpropagation.





- The model loses its ability to learn diverse and complex representations.
- **Solution**: Break the symmetry with random initialization.

Random Initialization

- May use random initialization to initialize the weights

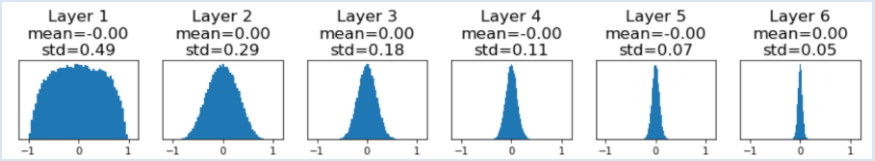
```
W = k * np.random.randn(F[l], F[l-1])
```

Use *k* to control the size of the random weights

- Good enough for small network:
  - No symmetry issue
- Issues when used for bigger network:
  - Sensitive to the size of the random weights or *k*.
  - May encounter vanishing gradient

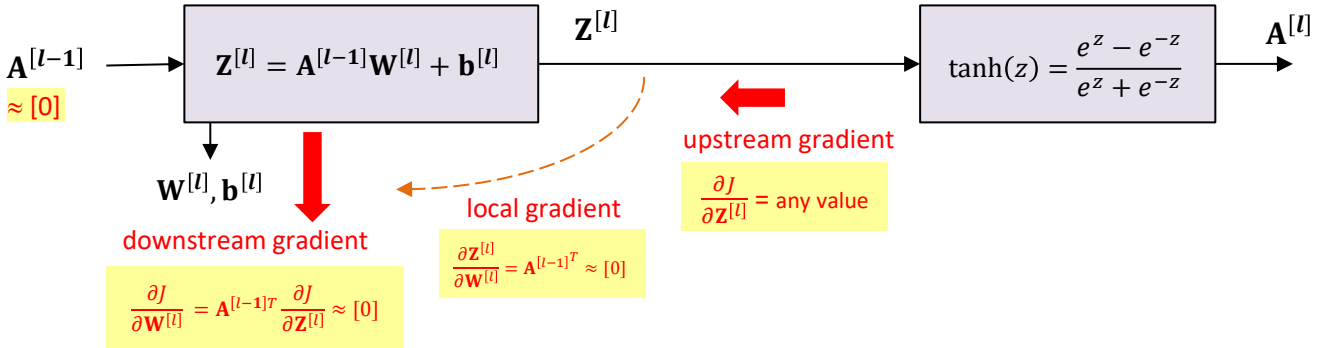
Issue with small random initialization

- For tanh/sigmoid activations, small random weights (`np.randn(size)*0.01`) results in mostly zero activation (mean ≈ 0 and variance ≈ 0) in deeper layers.



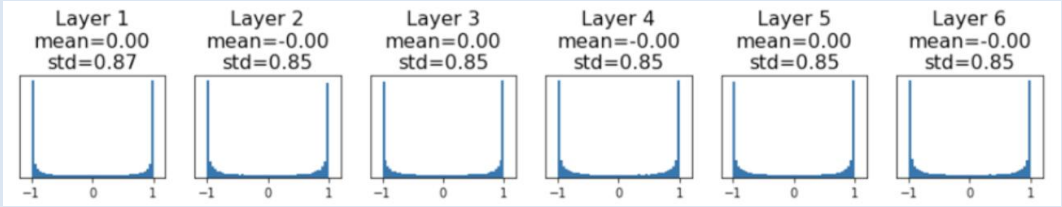
Activation statistics when using small random initialization for 6 layers each with 4096 neurons, tanh activations, batch size = 16,

- Activations ≈ 0 → local gradient ≈ 0 → gradients of weights ≈ 0 → no change to weights during training



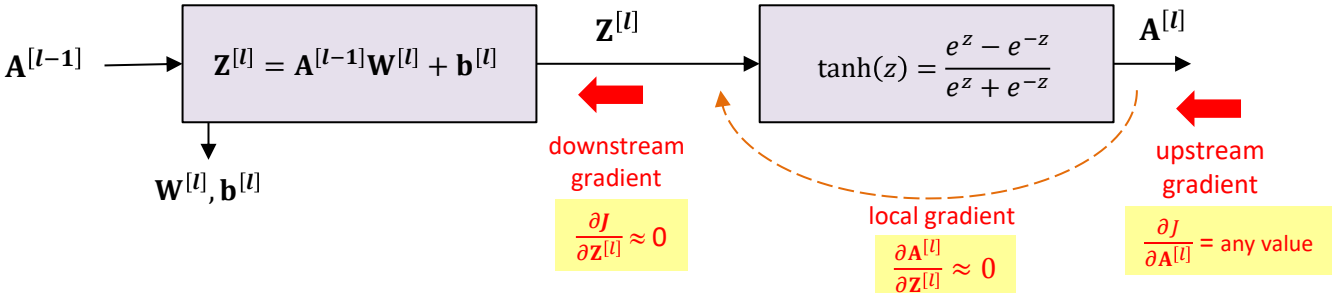
Issue with big random initialization

- For tanh or sigmoid activation, big random weights (`np.randn(size)*10`) causes most neuron to be saturated (≈ -1 or 1 for tanh). When that happens, variance ≈ 0.



Activation statistics when using big random initialization for 6 layers each with 4096 neurons, tanh activations, batch size = 16,

- Saturated neurons → local gradient ≈ 0 → downstream gradient ≈ 0



Xavier Initialization

[Glorot and Bengio, 'Understanding the difficulty of training deep feedforward neural networks", AISTAT 2010]

- Ensures that weights are just right, neither too large and small.
- Keeps variance of actions and gradients consistent across layers
- Uniform Distribution:

$$W \sim U\left[-\frac{\sqrt{6}}{\sqrt{F_{in}+F_{out}}}, \frac{\sqrt{6}}{\sqrt{F_{in}+F_{out}}}\right]$$

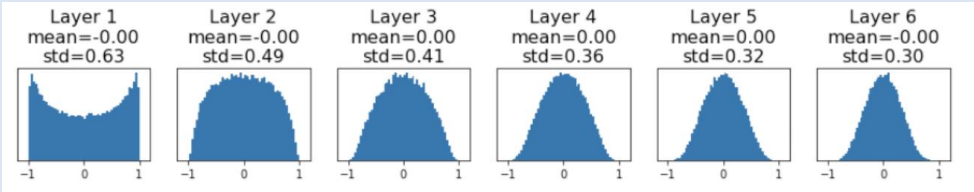
- W : Weight matrix
- F_{in} : No of input features to the layer
- F_{out} : No of output features of the layer

- Normal distribution:

$$W \sim \mathcal{N}\left(0, \frac{1}{F_{in}+F_{out}}\right)$$

Xavier Initialization

- Xavier Initialization preserves the variance of the activation values during forward pass and the gradient values during backpropagation.

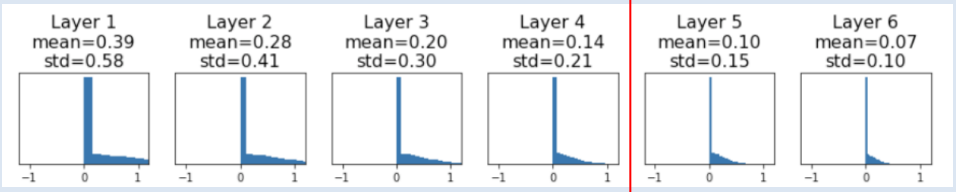


Activation statistics when using Xavier initialization for 6 layers each with 4096 neurons, tanh activations, batch size = 16

- Stable training: prevents vanishing and exploding gradient problems
- Works well on shallow and moderately deep networks with sigmoid and tanh activation functions

Issues with Xavier Initialization

- Xavier initialization was designed for linear activation and works decently well with sigmoid and tanh.
- Does not not very well for ReLU activation — most activations ≈ 0 and the variance ≈ 0 for deeper layers:



Activation statistics when using Xavier initialization for 6 layers each with 4096 neurons, relu activations, batch size = 16

- **Solution**: Kaiming initialization.

Kaiming Initialization

[He et al., "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification", ICCV 2015]

- Similar strategy for Xavier initialization but reformulated for ReLU activation.
- Uniform Distribution:

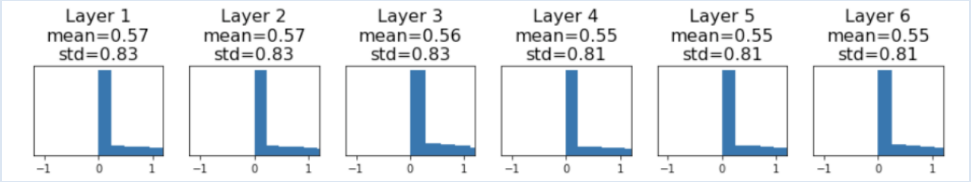
$$W \sim U\left[-\frac{\sqrt{6}}{\sqrt{F_{in}}}, \frac{\sqrt{6}}{\sqrt{F_{in}}}\right]$$

- W : weight matrix
- F_{in} : number of input features to the layer

- Normal distribution:

$$W \sim \mathcal{N}\left(0, \frac{2}{F_{in}}\right)$$

- Xavier Initialization preserves the variance of the activation values



Activation statistics when using Kaiming initialization for 6 layers each with 4096 neurons, relu activations, batch size = 16

Summary

- Initializing the weights:
  - Use Kaiming initialization (relu activations) or Xavier (other activation) when training from scratch
  - Best to initialize from pretrained model to take advantage of prior knowledge.
- Initializing the biases:
  - Common to initialize the biases to zero (no symmetry problem).
  - Can also use a small constant such as 0.01 for ReLU to ensure that all ReLU fires in the beginning and therefore propagate some values.

Normalization Layers

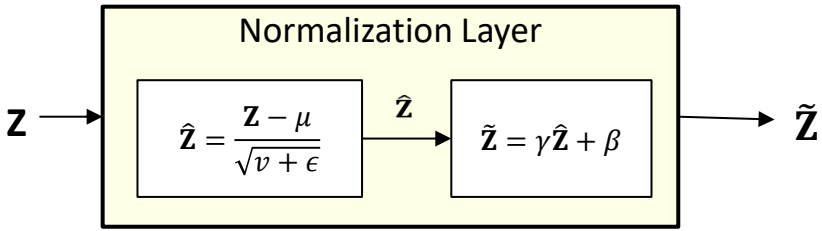
Normalization

- Standardize input distribution so that mean = 0 and std = 1.
- Stabilizes and accelerates training.
- General formulation for normalization:

$$\hat{\mathbf{Z}} = \frac{\mathbf{Z} - \mu}{\sqrt{v + \epsilon}}$$

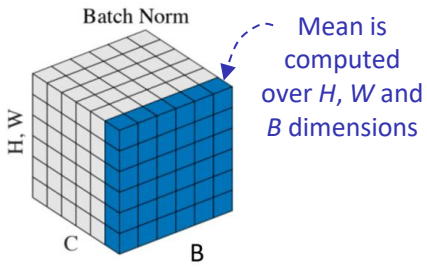
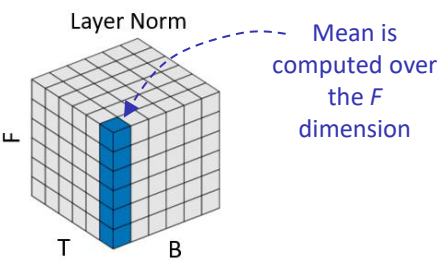
μ: mean of some dimensions
v: variance of some dimensions

- Normalization also provides the mechanism to undo normalization if required.



- If $\gamma = \sqrt{v + \epsilon}$ and $\beta = \mu$: no normalization where $\tilde{\mathbf{Z}} \approx \mathbf{Z}$
- If $\gamma = 1$ and $\beta = 0$: fully normalized where $\tilde{\mathbf{Z}} \approx \hat{\mathbf{Z}}$
- γ and β are network parameters that are learnt

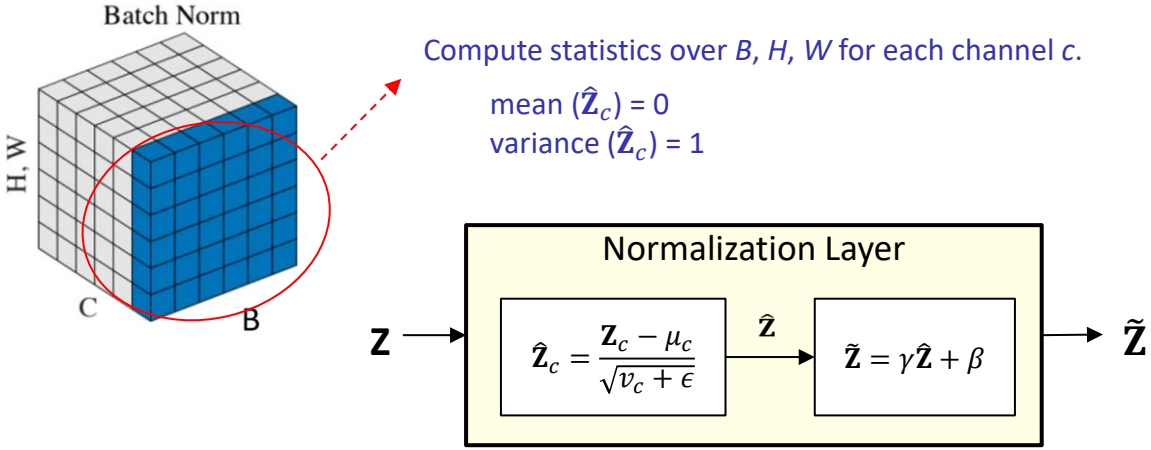
Normalization Layers (Image Data)

	BatchNorm	LayerNorm
		
Normalizes over	Batch and location dimensions (*B*, *H*, *W*) for each channel (*c*)	Normalize over feature dimensions (*F*,) for each timestep and sample (*b*, *t*)
Typical use	Good for CNN	Good for Transformer
Stability	Can be unstable for tiny batches	Very stable
Strength	Reduces internal covariate shift, reduces covariate shift between training and testing, accelerates CNN training	Stabilizes deep sequence/Transformer optimization by normalizing each feature vector independently

BatchNorm

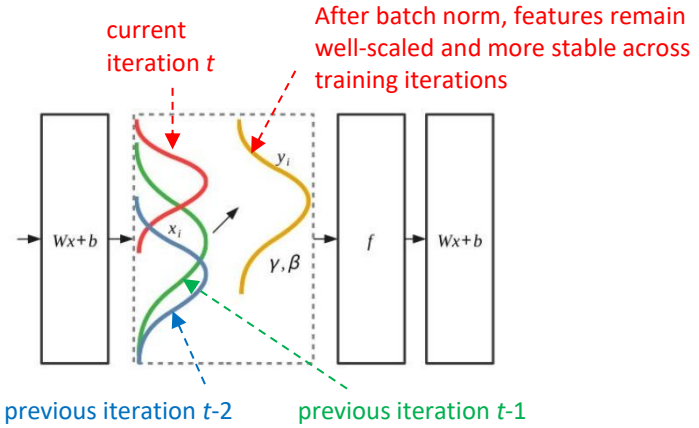
[Ioffe and Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, ICML2015]

- BatchNorm normalizes over (across) the batch (and spatial) dimensions per feature or channel.
- For each channel c, the normalized activations $\hat{\mathbf{Z}}_c$ have mean ≈ 0 and variance ≈ 1.



Robustness to Internal Covariate Shift

- BatchNorm makes the model more robust to internal covariate shift during training.
- **Internal Covariate shift** occurs during training when the input distribution of a layer changes as the parameters of earlier layers are updated. This causes unstable gradients and slow optimization.



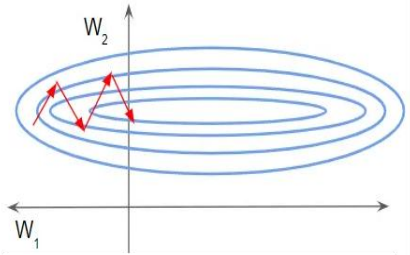
- BatchNorm keeps the layer activations well-scaled during training, which stabilizes gradient propagation, and enables faster, more reliable optimization

BatchNorm improves the cost surface

[Santurkar, S., et al., *How Does Batch Normalization Help Optimization?*. NeurIPS 2018]

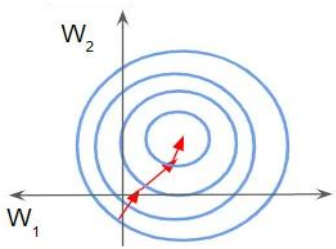
- Batch Normalization smooths the optimization landscape, making gradient descent easier.

Without BatchNorm:
(Ill-conditioned optimization)



Batch Norm

Without BatchNorm:
(Better-conditioned optimization)



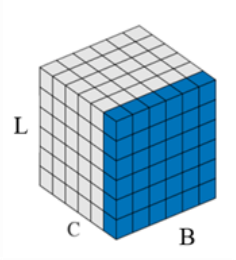
- Large features dominating gradient updates
- Narrow ravine loss landscape
- Gradient oscillation
- Slower convergence

- Layer activations kept on a similar scale
- More uniform loss landscape
- Smoother gradient descent
- Faster convergence

BatchNorm: Dimensionality

BatchNorm1D

(1D CNN)



$\mathbf{Z}$: $B \times C \times L$

BatchNorm1D

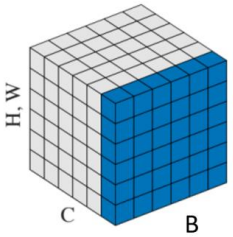
μ, σ: $(C,)$

γ, β: $(C,)$

$\hat{\mathbf{Z}}$: $B \times C \times L$

BatchNorm2D

(2D CNN)



$\mathbf{Z}$: $B \times C \times H \times W$

BatchNorm2D

μ, σ: $(C,)$

γ, β: $(C,)$

$\hat{\mathbf{Z}}$: $B \times C \times H \times W$

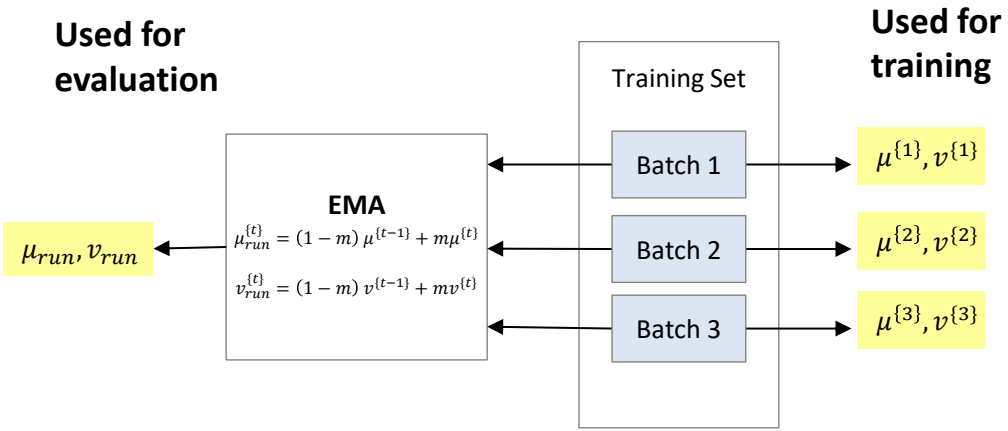
BatchNorm: Training vs Evaluation Mode

- **Training:**

  Compute mean and variance from the current mini-batch for each channel and update running (EMA) statistics.

- **Evaluation:**

  Normalize using the running mean and variance accumulated (EMA) during training (no batch statistics).



$\mu^{\{1\}}, v^{\{1\}}$: mean and variance of the mini-batch at iteration t.

BatchNorm: Where to put?

- More effective for Convolutional Neural Network. Less effective for sequence models.
- More effective to place BatchNorm after the FC or CONV layer and before activation. Drop the bias for the FC or CONV layer.

- For ResNet, place before adding the residual connection

- Do not use BN with dropout (noisy mean and variance after dropout)
- Works better with larger batch size for more stable statistics (at least 32 samples).

BatchNorm: Strengths

- Improves gradient flow, less vanishing gradient issues.
- Allows higher learning rates, faster convergence .
- Less sensitive to weight initialization.
- Acts as a form of regularization (Batch mean and variance adds noise to the weighted input)

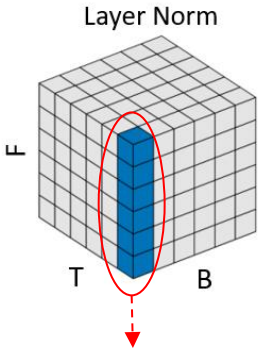
BatchNorm: Weaknesses

Batch Norm cannot handle sequence data well:

- **Variable Sequence Length**
  - Different amount of padding → Inconsistent normalization, different amounts of meaningful content.
- **Batch size dependence:**
  - Long sequence may limit batch size → small batch size → estimated statistics not reliable → unstable training, does not accurately represent the true distribution of specific sequence or position during inference.
- **Position-specific statistics**:
  - Tokens at different positions have different statistical properties
  - Mixing statistics across positions and sequences may destroy important positional information
- **Autoregressive problem:**
  - Disrupt temporal dependencies by mixing statistics from different time steps. Information from future time steps leaks to earlier ones.

LayerNorm

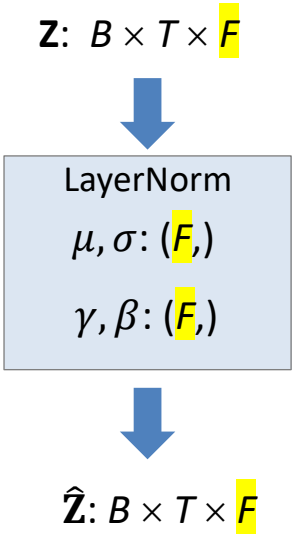
- Normalizes across the feature dimension (F) for each sample and time step (b, t).
- Stabilizes training and improves gradient flows



Compute statistics over F for each sample b and time step t

mean (*feature*) = 0

var (*feature*) = 1



- In PyTorch, `nn.LayerNorm` normalizes over the last `normalized_shape` dimensions. (`norm_layer = nn.LayerNorm(F)`)

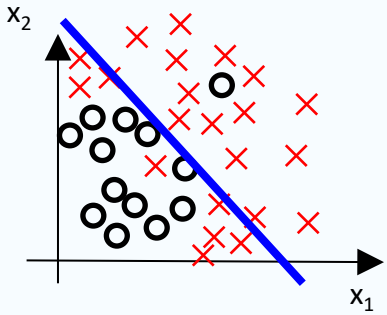
Why LayerNorm works for Sequential Models

- **Batch independent**: Works even with batch size = 1 (important for variable-length sequences or autoregressive sequences).
- **Stabilizes activations**: Keeps activations well-scaled within each layer during training.
- **Accelerates training**: Smoother loss landscapes, enables higher learning rates and faster convergence.
- **Improves gradient flow**: Helps reduce vanishing / exploding graidents in deep or recurrent networks.

Underfitting and Overfitting Issues

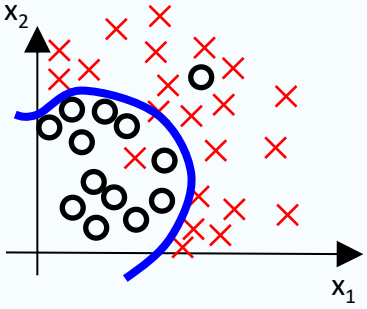
Overfitting & Underfitting

Underfitting
(High Bias)



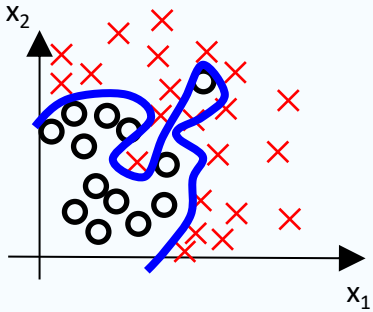
- Bad training accuracy
  Bad testing accuracy
- Low representation capacity (classifier is too simple)
- Not caused by the size of training set

Just right



- Good training accuracy
  Good testing accuracy
- Classifier is at the right complexity
- Sufficient training samples

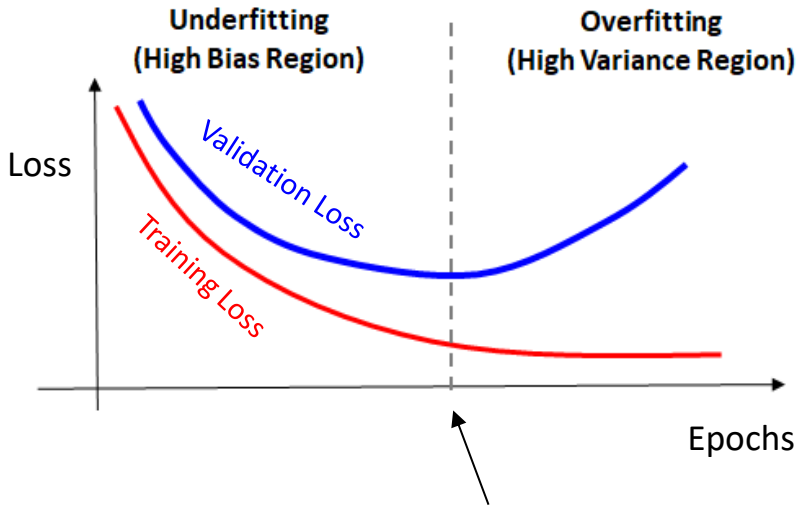
Overfitting
(High Variance)



- Good training accuracy
  Bad test accuracy
- Classifier is too complex (high representation power) for the task
- Not sufficient training samples

Diagnosing your training

- Compare the training and validation losses to check for overfitting or underfitting issues

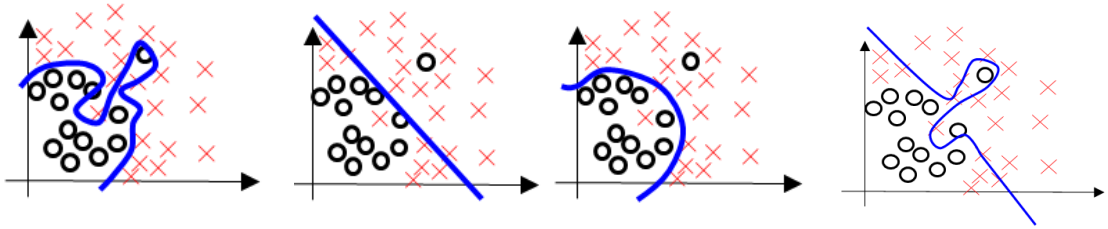


Ideally, stop the training at the minimum point of validation loss (early stopping)

Observing training and validation accuracies

- To detect overfitting or underfitting, compare the training and validation accuracies
- Example (assume human or state-of-the-art accuracy ≈ 80%) :

Training acc	79%	65%	78%	65%
Validation acc	65%	66%	79%	55%



Overfitting (Does not generalize well)

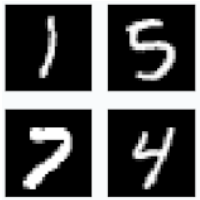
Underfitting (Model is too simple)

No overfitting No underfitting (Works well)

Overfitting + Underfitting (at different regions)

Factors affecting overfitting

- Complexity of the model
  - Larger neural networks have more parameters and higher capacity → more prone to overfitting if data is limited.
- Size of training set
  - Sufficient?
  - Representative?
  - Matches the target domain?
  - Insufficient or non-representative training set leads to overfitting
- Complexity of task
  - Complex tasks require larger and more diverse training set to avoid overfitting.



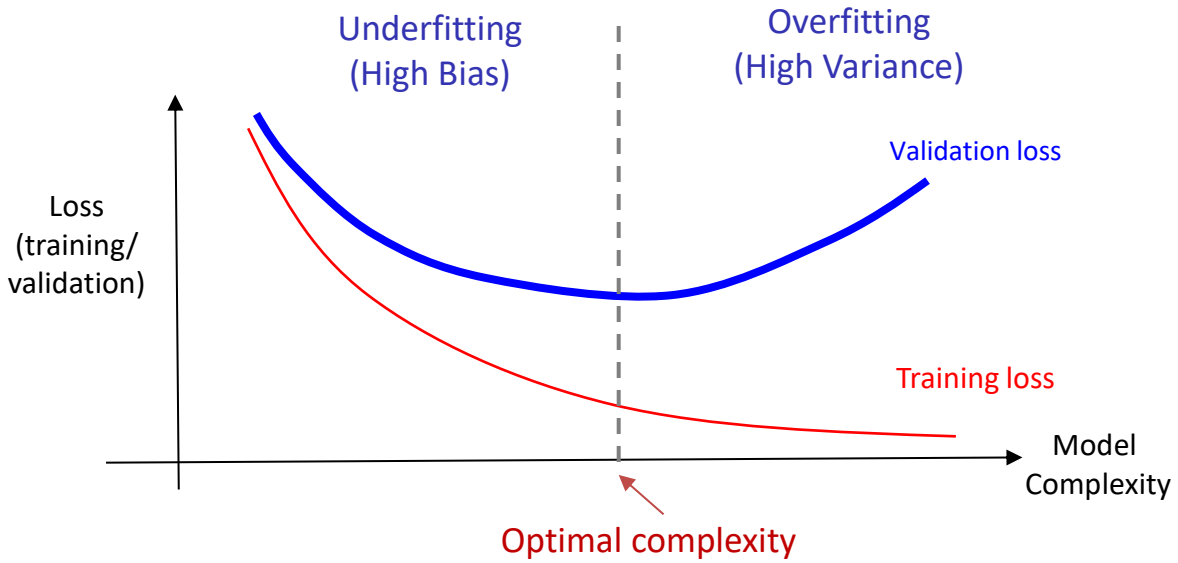
Easy

Moderate

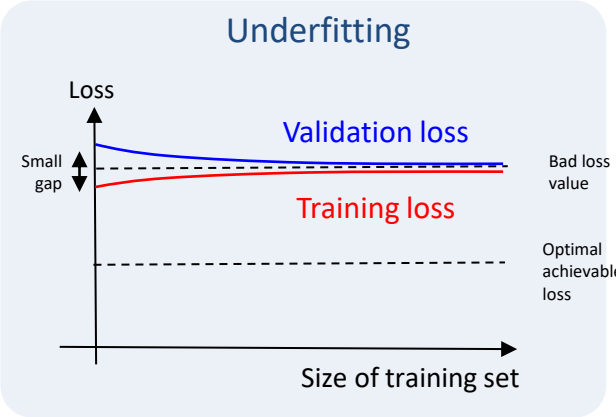
Complex

Effect of model complexity

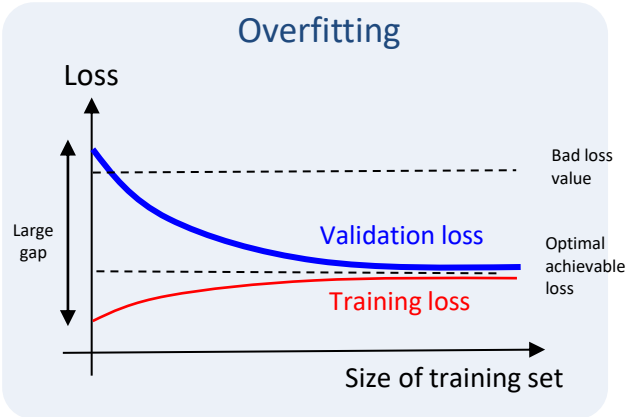
- An overly complex model may cause overfitting, while an overly simple model may cause underfitting.
- Model complexity is affected by network architecture (e.g., depth/width) and regularization strength.
- Select the model that gives the lowest validation loss.



Effect of training set size



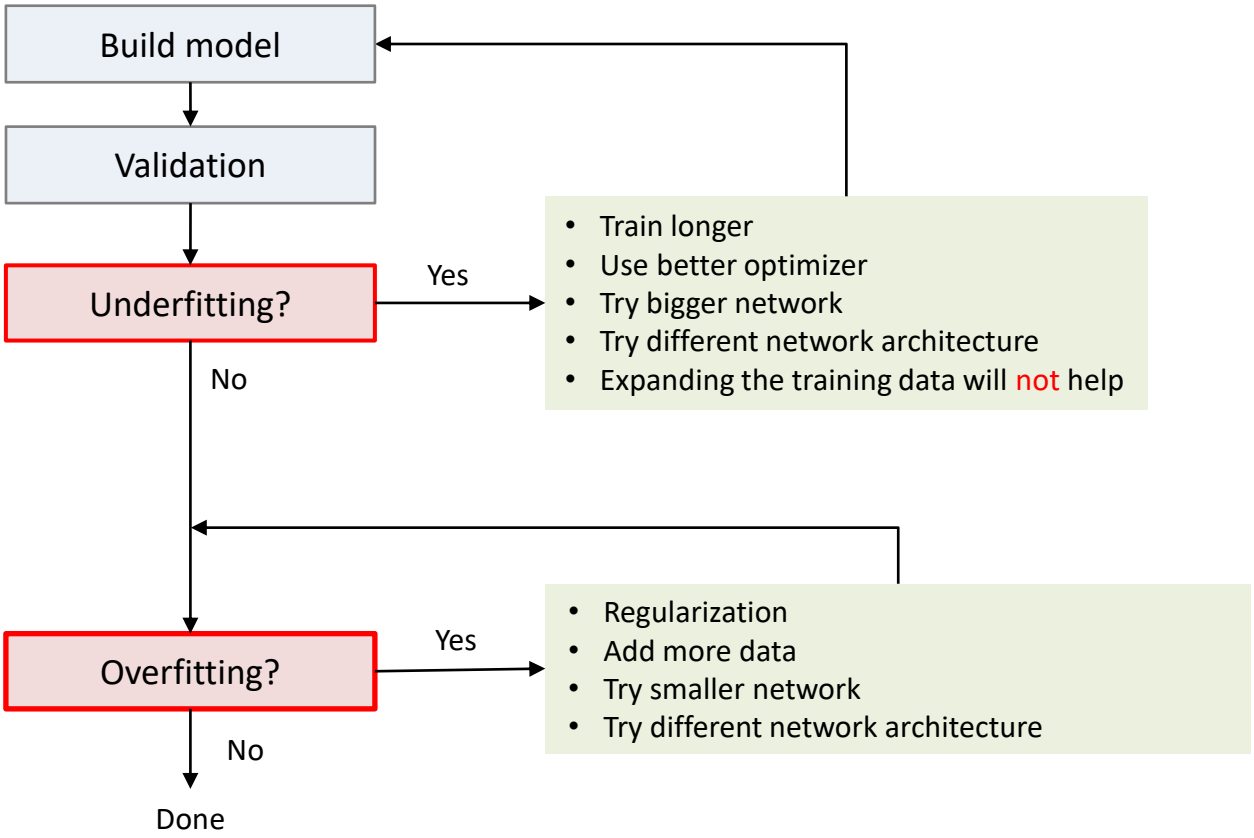
- Training and validation losses are both high, and their **gap** is small.
- Increasing training set size does not significantly reduce the error



- With a small training set, training loss is low, but validation loss is high. The gap between them is large.
- Gap narrows when training set increases.
- Increasing the training set size helps to reduce overfitting.

Notes: Training loss may increase slightly as the training set grows because the model must fit a more diverse set of samples.

Workflow for handling overfitting and underfitting



Underfitting and Overfitting Issues:

Regularization

Regularization

- Regularization refers to techniques that reduce overfitting and improves generalization to unseen data.
- Three ways to perform regularization
  - Loss-based regularization (weight decay)

Regularization Loss

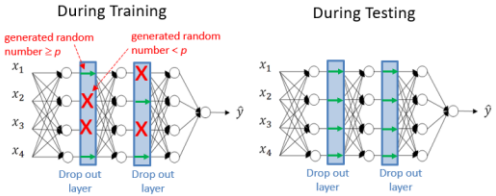
$$J_{reg} = J + \frac{\lambda}{2m}\sum_{i=1}^{L}\left\|\mathbf{W}^{[i]}\right\|_2^2$$

Adding regularization loss penalizes large weights → smoother / less complex model → reduced overfitting

  - Data-based regularization (data augmentation)

Increases effective training data using label-preserving transformation

  - Architecture-based Regularization (dropout)



Randomly drop neurons during training. Prevents co-adaptation and acts like training an ensemble of sub-networks.

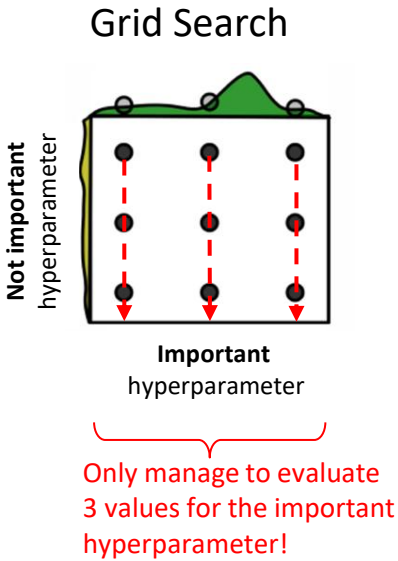
Hyperparameter Tuning

Hyperparameter Tuning

- How do we set the hyperparameters?
  - **Minibatch**: batch size
  - **Optimizer type**: Adam, SGD, RMS, etc.
  - **Optimizer settings**: learning rate, momentum
  - **Scheduler**: warmup steps, learning rate
  - **Network architecture**: number of layers, number of units per layer, activation functions, dropout rate
  - **Regularization**: weight decay
- The best hyperparameter setting is determined empirically by trying out different process. Four common search method:
  1. Manual
  2. Grid Search
  3. Random search
  4. Bayesian model-based optimization

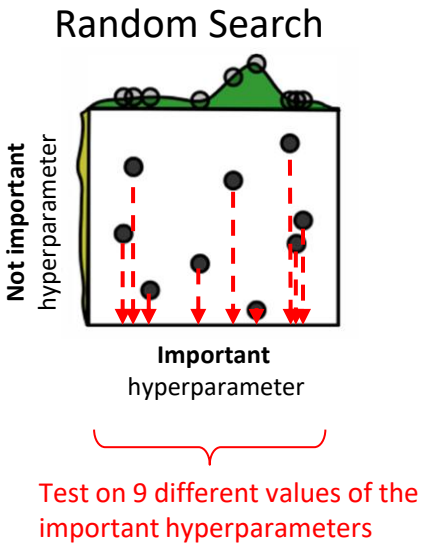
Grid Search

- Grid search:
  1. Defines some grid of parameters
  2. Build a model and evaluate for each combinations
  3. Choose the setting with the best result
- Essentially a brute force method
  - Trains v^h models (*v*: number of parameters in each hyperpameter, *v*: number of hyperparameters)
  - As number of parameters *v* for each of the *h* hyperparameters increases, cost of grid search increases exponentially
- Most time may be spent searching on not important hyperparameters.



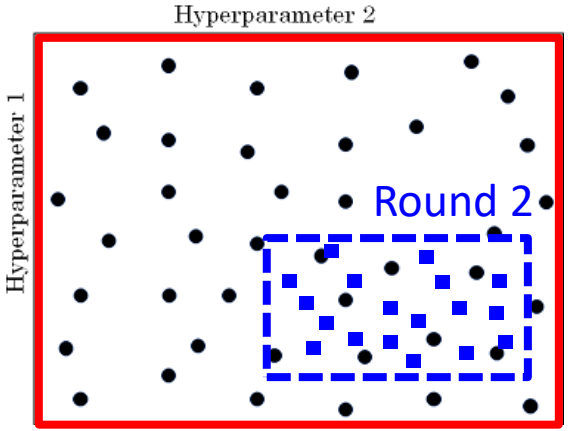
Random Search

- **Random Search** evaluates on randomly chosen points instead of points on a grid
- Evaluate as many values for all hyperparameters for the same number of tests as Grid Search
- Empirically very effective
- Sometimes, may miss the good regions



Efficient Hyperparameter Tuning

1. Finetune on a subset of the training set (fewer sample)
2. Perform a coarse-to-fine tuning:
   - **Coarse tuning**: use fewer epochs to get rough idea of the working range
   - **Finer tuning**: use longer epochs, then perform finer search
   - Repeat if necessary



3. Lastly, train the model on the full training set with the best hyper-parameter with the maximum number of epoch.

Example:

First, perform a coarse random search for a small #epochs

Sample 100 points
Regularization strength (reg) = 1e-6 to 1e5 (coarse range)
Learning rate (lr) = 1e-6 to 100 (coarse range)
Number of epochs = 5 (small #epoch)

```
val_acc:  0.412000,  lr: 1.405296e-04,  reg:  4.793564e-01  (1/100)
val_acc:  0.214000,  lr: 7.231888e-06,  reg:  2.321281e-04  (2/100)
val_acc:  0.208000,  lr: 2.119571e-06,  reg:  8.011857e+01  (3/100)
val_acc:  0.196000,  lr: 1.551131e-05,  reg:  4.374936e-05  (4/100)
val_acc:  0.079000,  lr: 1.753300e-05,  reg:  1.200424e+03  (5/100)
val_acc:  0.223000,  lr: 4.215128e-05,  reg:  4.196174e+01  (6/100)
val_acc:  0.441000,  lr: 1.750259e-04,  reg:  2.110807e-04  (7/100)
val_acc:  0.241000,  lr: 6.749231e-05,  reg:  4.226413e+01  (8/100)
val_acc:  0.482000,  lr: 4.296863e-04,  reg:  6.642555e-01  (9/100)
val_acc:  0.079000,  lr: 5.401602e-06,  reg:  1.599828e+04  (10/100)
val_acc:  0.154000,  lr: 1.618508e-06,  reg:  4.925252e-01  (11/100)
```

Good result for:
$1e\text{-}04 \leq lr \leq 1e\text{-}03$
$1e\text{-}04 \leq reg \leq 0$

Next: do finer search

[Source: Stanford cs231n]

Example:

Next, run finer search for more epochs

Sample 100 points
Regularization strength (reg) = 1e-4 to 0 (finer range)
Learning rate (lr) = 1e-4 to 1e-3 (finer range)
Number of epochs = 100 (bigger #epoch)

```
val_acc: 0.527000, lr: 5.340517e-04, reg: 4.097824e-01, (0 / 100)
val_acc: 0.492000, lr: 2.279484e-04, reg: 9.991345e-04, (1 / 100)
val_acc: 0.512000, lr: 8.680827e-04, reg: 1.349727e-02, (2 / 100)
val_acc: 0.461000, lr: 1.028377e-04, reg: 1.220193e-02, (3 / 100)
val_acc: 0.460000, lr: 1.113730e-04, reg: 5.244309e-02, (4 / 100)
val_acc: 0.498000, lr: 9.477776e-04, reg: 2.001293e-03, (5 / 100)
val_acc: 0.469000, lr: 1.484369e-04, reg: 4.328313e-01, (6 / 100)
val_acc: 0.522000, lr: 5.586261e-04, reg: 2.312685e-04, (7 / 100)
val_acc: 0.530000, lr: 5.808183e-04, reg: 8.259964e-02, (8 / 100)
val_acc: 0.489000, lr: 1.979168e-04, reg: 1.010889e-04, (9 / 100)
val_acc: 0.490000, lr: 2.036031e-04, reg: 2.406271e-03, (10 / 100)
val_acc: 0.475000, lr: 2.021162e-04, reg: 2.287807e-01, (11 / 100)
val_acc: 0.460000, lr: 1.135527e-04, reg: 3.905040e-02, (12 / 100)
val_acc: 0.515000, lr: 6.947668e-04, reg: 1.562808e-02, (13 / 100)
val_acc: 0.531000, lr: 9.471549e-04, reg: 1.433895e-03, (14 / 100)
val_acc: 0.509000, lr: 3.140888e-04, reg: 2.857518e-01, (15 / 100)
val_acc: 0.514000, lr: 6.438349e-04, reg: 3.033781e-01, (16 / 100)
val_acc: 0.502000, lr: 3.921784e-04, reg: 2.707126e-04, (17 / 100)
val_acc: 0.509000, lr: 9.752279e-04, reg: 2.850865e-03, (18 / 100)
val_acc: 0.500000, lr: 2.412048e-04, reg: 4.997821e-04, (19 / 100)
val_acc: 0.466000, lr: 1.319314e-04, reg: 1.189915e-02, (20 / 100)
val_acc: 0.516000, lr: 8.039527e-04, reg: 1.528291e-02, (21 / 100)
```

Best cross-validation result

Should continue refining since all top accuracies are at the edge of the evaluated learning rate range (~1e-4). Can try between 1e-4 and 1e-6

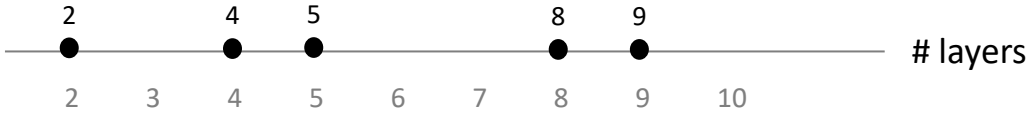
Direct sampling of hyperparameters

- Some hyperparameters can be directly sampled in the original space
- Example:
  - Number of units: $F^{[l]}$ = 50, .., 100

```
np.random.randint (low=50, high=101, size=5)
```

  - Number of layers: L = 2...10

```
np.random.randint (low=2, high=11, size=5)
```

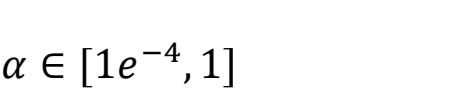


Sampling Bias in Linear Scale for Multi-Scale Hyperparameters

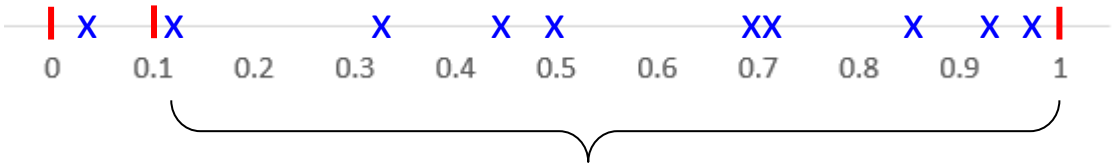
- Hyperparameters spanning multiple scales (e.g., learning rate, weight decay) are not suitable to be sampled uniformly in linear scale.

Learning rate $\alpha \in [1e^{-4}, 1]$

np.random.uniform (1e-4, 1, size = 10)



0.96286155, 0.01234344,
0.12309451, 0.50956422,
0.70601849, 0.86635589,
0.34336656, 0.74546455,
0.43543534, 0.93433333



☹ Most samples fall in the higher range (0.1 to 1)
☹ The lower range (1e-4 to 0.1) is under-sampled

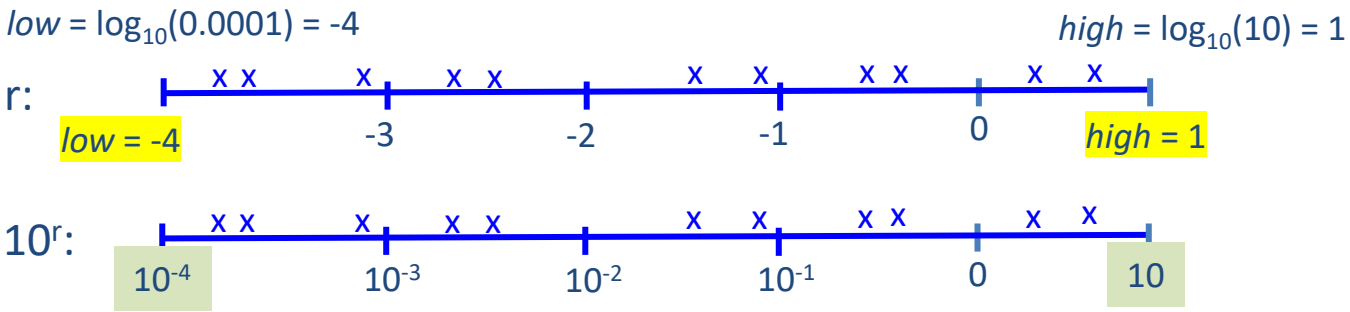
- For hyperparameters that crosses multiple scales, sample on a log scale.

Sampling in log scale

- For hyperparameters that span across different scales, sample in log space

Example: sample the learning rate between 0.0001 and 10

```
low  = np.log10 (0.0001)                          # convert to log space
high = np.log10(10)
log_lr = np.random.uniform(low, high, size = 10)  # sample in log space
lr = 10**log_lr                                   # convert back to original space
```



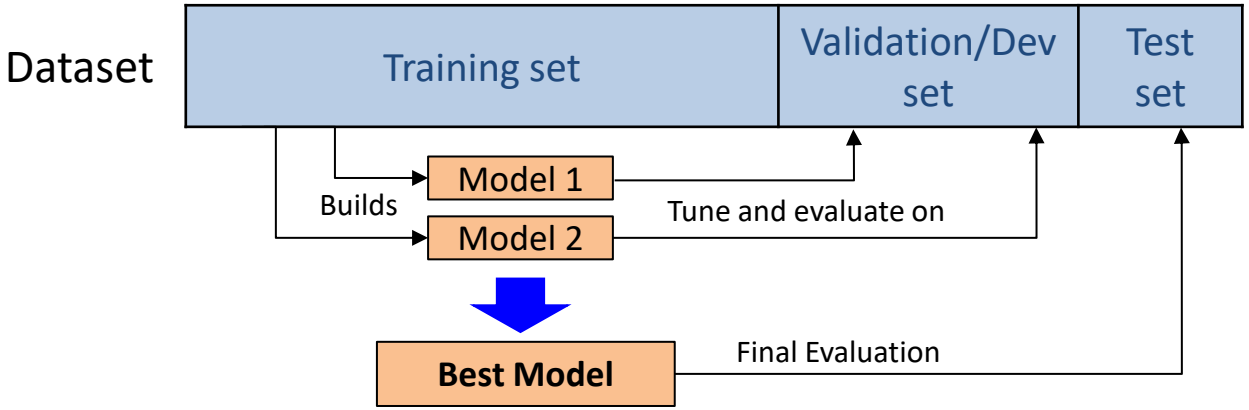

```
lr =  ((0.33850934,  0.01159559, 0.01392664, 0.05365572, 0.00223170,
        0.00034188, 0.00035015, 0.00130088, 0.00316376))
```

Prioritizing the hyperparameters

- Some hyperparameters are more important than others
- Most important:
  - α (learning rate)
- 2nd in importance:
  - β (momentum hyperparameter, normally ~0.9 works well)
  - #hidden units
  - mini-batch size
- 3rd in importance:
  - #layers
  - learning rate decay
- Normally no need for tuning:
  - β_1, β_2, ε: (Adam hyperparameters, set to 0.9, 0.999, 10^{-8})

Reliability of Reported Performance

Training vs Validation vs Test Set



Training set	**Validation set**	**Testing set**
• Training accuracy tends to be over-optimistic	• Used for hyperparameter tuning and model selection • Risk of overfitting the validation set • Often incorrectly referred to as test set	• Used to evaluate the generalization performance of the model • Never go back to fine-tune the model • Sometimes not available

Distribution of Training, Validation and Test Data

- Validation and test sets must reflect the deployment distribution; otherwise evaluation is misleading.
- Distribution shift occurs when the validation or test set does not match the target (real-world) the distribution.
- Training set can be larger or mixed.

Training & Evaluation	Targeted domain
High resolution images of cats	Low resolution images of cats
Movie videos	Surveillance videos
Simple image with single object	Complex image with cluttered background with many objects
Training on only a few breeds of dogs	All breeds of dogs and cats

Sizes of the set

- Small to medium big-data, e.g., 100-10000 examples

- Big dataset, e.g., 1 million samples

10000 samples

2500 samples

Reliability of Reported Performance

- Fair Comparison:
  - Tune the hyperparameters separately for each architecture:
    - Do not use a single default hyperparameter for all evaluated architectures.
    - Different architectures have different optimal settings.
    - Use the same tuning effort (same search space / number of trials)
  - Compare architectures with similar parameter count / FLOPs
    - Otherwise, improvement may come from model size, not architecture.
  - Use the same training setup
    - Same data splits
    - Same data augmentation.
    - Same training steps or epochs (when efficiency is considered)
- Split validation and testing (if possible)
  - Tune on validation set
  - Evaluate only once on test set. Do not optimize the test accuracy.
  - Report the test accuracy (more realistic), not the training accuracy (too optimistic).
  - For small dataset, may not have a test set. Do k-fold cross-validation instead.

Reliability of Reported Performance

- Random performance:
  - Result may vary between runs due to randomness during training. Cause: data partitioning, batch data sampling, initialization, dropout layer, etc.
  - Report average accuracy over multiple (e.g., 3) runs.
  - May report the standard deviation over the runs to report the degree of spread of performance of model.
  - Use cross-validation when data is limited.
  - Report confidence interval (e.g., mean ± 95% CI) or perform statistical test when differences are small.
- Avoid data leakage
  - No preprocessing using full dataset statistics.
  - Not test data used during tuning.
  - Split before feature engineering.
- Ensure reproducibility
  - Expected in research and industry
  - Fix random seeds
  - Report dataset version, preprocessing, hardware / framework, save configuration file, model files

Reliability of Reported Performance

- Proper Baseline
  - Compare against strong baselines such as previous state-of-the-art.
  - Trivial to compare against simple and old model (logistic regression, small CNN, etc.).
  - Perform ablation study: remove components to show contribution.
- Proper evaluation metrics and analysis
  - Use appropriate metric for the task (e.g., accuracy, F1, AUC, precision/ recall).
  - For imbalanced dataset, avoid accuracy. Instead use F1, precision / recall, or balanced metrics.
  - Report multiple metrics to analyze the results from different perspectives.
  - Interpret results critically. Highlight key findings. Go beyond numbers — explain *why* the model performs as observed.
- Monitor the training
  - Plot training vs validation curves
  - Helps to detect underfitting, overfitting and early stopping point

Thank you

for your attention